

Accés à une BD distante avec android en utilisant une Application Web



Plan

I. INTRODUCTION ---- 1

II. APPLICATION WEB ---- 1

III. APPLICATION HYBRIDE ---- 2

IV. APPLICATION NATIVE ---- 3

IV.1. ANDROID – APPELS RESEAU AVEC VOLLEY ---- 3

IV.1.1. Encore une nouvelle API réseau ? ---- 3

IV.1.2. Création de la Request ---- 3

IV.2. PERMISSION INTERNET ---- 4

IV.3. APPEL DE LA BIBLIOTHEQUE DANS LE GRADULE ---- 4

IV.4. LE FORMAT JSON ---- 4

IV.5. JSONOBJECT ---- 4

IV.6. JSONARRAY ---- 4

IV.7. EXEMPLE1 ---- 5

IV.8. EXEMPLE2 ---- 6

Références

[1] http://blog.xebia.fr/2013/07/08/android-appels-reseau-avec-volley/

[2] https://developer.android.com/training/volley/index.html

[3] https://www.androidtutorialpoint.com/networking/android-volley-tutorial/

I. Introduction

Android ne peut pas communiquer directement avec une base de données distante. Une application tierce doit s'insérer entre l'application android et le SGBD, cette application reçoit les requêtes de l'application android pour les envoyer au SGBD, par la suite il récupère le résultat et envoi la réponse à l'application android. Cette application tiers peut être une application web ou un service web ou autre.

Dans ce cours, l'application tierce est une application web. Pour appeler cette application, il existe trois solutions d'application mobile :

- utiliser une application web,
- utiliser une application hybride,
- utiliser une application native.

II. Application Web

Dans cette solution aucune ligne de code n'est écrite. L'utilisateur lance le navigateur du Smartphone puis tape l'adresse de l'application Web. Cette solution nécessite une connexion internet et elle fonctionne sur tous les Smartphones.

III. Application hybride

Cette solution consiste à ajouter un WebView à l'activité et appeler l'adresse de l'application Web.
Il faut ajouter la permission Internet à AndroidManifest .xml et aussi l'option usesCleartextTraffic.

```
<uses-permission android:name="android.permission.INTERNET" />
<application

    android:usesCleartextTraffic="true">
```

Exemple :

```
class MainActivity : AppCompatActivity() {
    private lateinit var amb: ActivityMainBinding;
    private lateinit var webV1: WebView
    // Correction de l'espace accidentel dans l'URL
    private val url = "http://192.168.1.19/JQM_PHP_Bib/bib/index.html"
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        initGraphique()
    }
    private fun initGraphique() {
        amb= ActivityMainBinding.inflate(layoutInflater)
        setContentView(amb.root)
        webV1 = findViewById(R.id.webV1)
        // Empêche l'ouverture du navigateur externe lors d'un clic sur un lien
        webV1.webViewClient = WebViewClient()
        // Configuration des paramètres du WebView via la fonction 'apply' de Kotlin
        webV1.settings.apply {
            loadsImagesAutomatically = true
            javaScriptEnabled = true
            domStorageEnabled = true // Recommandé pour les applications web modernes
        }
        webV1.loadUrl(url)
    }
}
```

IV. Application native

Cette solution consiste à écrire une application native qui envoie des requêtes http à l'application web et reçoit les réponses dans un format xml, json ou autre …

Dans les cas à traiter, On suppose que les pages de l'application Web utilisent la méthode "POST" et que le format des réponses est "JSON".

IV.1. Android – Appels réseau avec Volley

IV.1.1. Encore une nouvelle API réseau ?

Il existe plusieurs moyens à notre disposition sur Android pour faire des appels réseaux [1]:

- On peut citer par exemple HttpURLConnection qui souffre de certains bugs sur les versions d'Android antérieures à Gingerbread (Android 3.0), dont un nous obligeant à désactiver le KeepAlive.
- Il y a également Apache HTTP Client qui fonctionne mieux pour les appareils pre-Gingerbread mais qui comporte également quelques bugs et n'est pas vraiment supporté par l'équipe Android (l'API étant trop grande et difficile à améliorer sans casser la compatibilité).

Que ce soit avec HttpURLConnection ou HTTPClient, il est nécessaire de faire les appels réseaux dans un thread différent du thread principal qui est dédié à l'interface utilisateur.

Pour cela, on utilise généralement sur Android ce qu'on appelle des AsyncTask, qui permettent d'exécuter du code en tâche de fond et de récupérer le résultat sur le thread principal pour nous permettre de mettre à jour l'interface utilisateur.

Ces AsyncTask ont également plusieurs problèmes, le plus important étant qu'ils peuvent être à l'origine de fuites de mémoire.

De plus, contrairement à la croyance populaire, les AsyncTask s'exécutent par défaut en série et non en parallèle, ce qui peut poser des problèmes de performance.

C'est pour pallier tous ces problèmes que Volley a vu le jour :

- Volley fonctionne avec un mécanisme de thread pool configurable
- Il supporte la priorisation des requêtes (par exemple, on peut vouloir définir une priorité haute pour le téléchargement de données importantes comme les informations du client, et une priorité plus faible pour le téléchargement d'images)
- Les résultats des requêtes sont automatiquement mis en cache disque et mémoire
- On peut très facilement annuler des requêtes en cours, afin d'éviter qu'elles ne soient exécutées pour rien
- Volley supporte nativement le JSON, les images, et le texte brut. Il est possible de rajouter du support à d'autres types de données facilement.
- Enfin, on a la possibilité d'implémenter ses propres mécanismes de custom retry / backoff

Attention toutefois, Volley n'est pas forcément la solution à tous vos problèmes réseaux !

Même s'il a été pensé pour être très performant sur toutes les opérations de type RPC, il est déconseillé d'utiliser Volley pour du téléchargement de données très importantes, comme les opérations de streaming ou de téléchargement de médias audio/vidéo [1].

IV.1.2. Création de la Request

Nous allons maintenant créer notre ListActivity qui appellera les serveurs REST de GitHub à l'aide de Volley pour récupérer la liste des membres de l'organisation Xebia-France

Lorsque l'on souhaite faire une requête HTTP avec Volley, il suffit d'instancier objet héritant de Request, en lui spécifiant les comportements à exécuter en cas de réussite (dans notre cas, afficher la liste des utilisateurs récupérée) et en cas d'échec (afficher un message d'erreur).

Il faudra ensuite envoyer cette Request à la RequestQueue que l'on a initialisé précédemment pour qu'elle soit exécutée.

Volley fournit par défaut les classes suivantes héritant de Request :

- StringRequest qui retourne une String à partir d'une URL
- ImageRequest, qui retourne un Bitmap
- JsonObjectRequest, qui retourne un JSONObject
- JsonArrayRequest, qui retourne un JSONArray

Il est évidemment possible de créer ses propres classes héritant de Request, afin que Volley fasse un traitement plus adapté à nos besoins.

Ici, nous souhaitons recevoir un objet de type JSONArray, nous allons donc lancer une JsonArrayRequest.

IV.2. Permission Internet

Dans AndroidManifest.xml, ajouter :

```
<uses-permission android:name="android.permission.INTERNET" />
```

IV.3. Appel de la bibliothèque dans le gradule

Dans build.gradule de App, ajouter dans dependecies :

```
implementation("com.android.volley:volley:1.2.1")
```

IV.4. Le format JSON

JSON (JavaScript Object Notation) est un format léger d'enregistrement et d'échange de données. JSON est indépendant du langage de programmation, il est auto descriptif et simple à comprendre.

Les règles syntaxiques de JSON sont :

- Les données sont des paires nom/valeur séparés par " :"
- Les données sont séparées par des virgules ","
- Les accolades délimites les objets "{}"
- Les crochets délimites les tableaux "[]"

Une valeur JSON peut être :

- Un nombre (entier ou réel)
- Une chaîne de caractère (entre "")
- Un booléen (true ou false)
- Un tableau (entre [])
- Un objet (entre {})
- null

Exemple :

```
{"liste":
   [
       {"nom":"Mohamed", "prenom":"Ben Saleh",   "age" :54},
       {"nom":"Saleh",   "prenom":"Ben Salem",   "age" :63},
       {"nom":"Salem",   "prenom":"Ben Mohamed", "age" :65}
   ]
}
```

Cet exemple contient un objet JSON qui contient un tableau "liste". "liste" contient trois objets, le premier a pour nom "Mohamed", pour prenom "Ben Saleh", pour age "54"…

IV.5. JSONObject

Cette classe modélise un objet JSON. Parmi les méthodes de JSONObject :

- *JSONObject(String jsonObjectString)*
- *String getString(String name), int getInt(String name)…*
- *JSONObject getJSONObject(String name)*
- *JSONArray getJSONArray(String name)*

IV.6. JSONArray

Cette classe modélise un tableau JSON. Parmi les méthodes de JSONArray :

- *int length() : retourne la taille du tableau*
- *JSONObject getJSONObject(int index)*

- *JSONArray getJSONArray(int index)*

IV.7. Exemple1

ListeLivres.php ne prend aucun paramètre, elle retourne un objet JSON qui contient tous les livres comme suit :

```
{
  "liste":
     [
       {"id":"1","titre":"t1","nbPge":"110",},
       …
       {"id":"9","titre":"t9","nbPge":"190",}
    ]
}
```

```
private fun remplir() {
        val queue: RequestQueue = Volley.newRequestQueue(this)
        val url = "${Config.JSON_BIB_URL}/liste/ListeLivres.php"

        val stringRequest = StringRequest(
            Request.Method.POST, url,
            Response.Listener { response ->
                try {
                    val r = JSONObject(response)
                    val a: JSONArray = r.getJSONArray("liste")
                    adpLivre.clear()
                    for (i in 0 until a.length()) {
                        val o: JSONObject = a.getJSONObject(i)
                        val id = o.getInt("id")
                        val titre = o.getString("titre")
                        val nbPage = o.getInt("nbPage")
                        val l = Livre(id, titre, nbPage)
                        adpLivre.add(l)
                    }
                } catch (error: JSONException) {
                    Toast.makeText(
                        this,
                        "Problème d'analyse JSON: ${error.message}",
                        Toast.LENGTH_LONG
                    ).show()
                }
            },
            Response.ErrorListener { e ->
                Toast.makeText(this, "Problème d'appel HTTP:
${e.message}", Toast.LENGTH_LONG)
                    .show()
            }
        )
        queue.add(stringRequest)
    }
```

IV.8. Exemple2

La page Ajout.php permet d'ajouter un livre avec le titre et le nbPage passés en paramètre. La réponse est un fichier JSON {" Reponse" : "SUCCES"} en cas de succès et {" Reponse" : "ECHEC"} en cas d'échec.

```
  private fun ajouterLivre() {
        val queue: RequestQueue = Volley.newRequestQueue(this)

        // Utilisation de votre URL (supposant que Config.URL est défini ou
votre constante)
        val url = "${Config.JSON_BIB_URL}/ajout/Ajout.php"


        val stringRequest = object : StringRequest(
            Request.Method.POST, url,
            Response.Listener { response ->
                try {
                    val r = JSONObject(response)
                    val etat = r.getString("ETAT")
                    if (etat == "SUCCES") {
                        amb.edTitre1.setText("")
                        amb.edNbPage1.setText("")
                        amb.edTitre1.requestFocus()
                        Toast.makeText(this, "Livre ajouté avec succès",
Toast.LENGTH_SHORT).show()
                    } else {
                        Toast.makeText(this, "Echec de l'ajout",
Toast.LENGTH_LONG).show()
                    }
                } catch (error: JSONException) {
                    Toast.makeText(this, "Problème d'analyse JSON:
${error.message}", Toast.LENGTH_LONG).show()
                }
            },
            Response.ErrorListener { e ->
                Toast.makeText(this, "Problème d'appel HTTP: ${e.message}",
Toast.LENGTH_LONG).show()
            }
        ) {
            override fun getParams(): Map<String, String> {
                val headers = HashMap<String, String>()
                headers["titre"] = amb.edTitre1.text.toString()
                headers["nbPage"] = amb.edNbPage1.text.toString()
                return headers
            }
        }

        queue.add(stringRequest)
    }
```